CSC490 Final Report

Team Members
Minh Le (1007894432), Prashanth Shyamala (1008819021),
Shaan Purewal (1008332939), York Ng (1009153577)

Abstract

Recorded lectures are increasingly important in modern higher education, yet their length and lack of structure make it difficult for students to locate specific explanations during review. Existing tools offer keyword search over transcripts or basic chapter labels but do not support semantic, query-driven retrieval of relevant video segments. In this paper, we present **LectureClip**, a system that automatically indexes lecture recordings through multimodal embeddings and enables students to retrieve and assemble timestamped video clips relevant to a natural-language query. LectureClip combines speech-to-text transcription, sentence-level chunking, text and visual embedding, and vector-based semantic search within a serverless pipeline on AWS. Retrieved segments are temporally aggregated into coherent clips and returned alongside a presigned video URL. We evaluate retrieval quality on a manually annotated set of university lectures using Recall@k, and ablate over two embedding models and three modality configurations (text-only, frame-only, and text+frame). Our results show that combining text and frame embeddings with the Jina-clip-v2 model yields the strongest overall retrieval performance, and we identify future directions for improving clip coherence through alternative chunking strategies and learned ranking models. Our code implementation can be found at: https://github.com/UofT-CSC490-W2026/LectureClip-App

1 Introduction

Lecture recordings are ubiquitous in post-secondary education, with institutional learning management systems (LMSs) hosting thousands of hours of video per term. Yet these recordings remain difficult to navigate: a typical lecture spans 50–120 minutes, is labelled only by date or course, and offers no structured way to jump to where a concept is defined, worked through, or revisited.

In practice, students resort to manual scrubbing until they recognize the topic of interest on a slide or in the instructor's speech. Research on video-based learning shows that efficient segment-level navigation is a key factor in the effectiveness of recorded lectures as a study aid [9]. When this navigation is slow, students either abandon the recording or spend disproportionate time searching rather than learning.

Existing commercial tools address part of this problem. Panopto's Smart Search [17] indexes speech transcripts and on-screen text via OCR for keyword lookup with timestamped results. NoteGPT [15] generates chapter labels and summaries for uploaded videos. However, both rely on lexical or shallow matching—a search for "eigenvalue decomposition" will miss a segment where the instructor says "we can factor the matrix this way" without using the exact term—and return isolated timestamps rather than assembling results into a coherent, self-contained explanation, leaving students to manually evaluate and sequence matches.

On the research side, multimodal video understanding has advanced rapidly. Vision-language models such as ColPali [8] match natural-language queries directly to visual slide content, bypassing OCR entirely; Video-RAG [20] proposes retrieval-augmented generation pipelines that chunk long videos and retrieve relevant clips for complex queries; and Twelve Labs [21] offers a commercial API for semantic moment retrieval. However, none of these efforts yet target the specific structure of academic lectures, which is predominantly static (slides or a whiteboard), rich in domain notation, and organized around pedagogical sequences of definition, example, and application.

In this paper we present **LectureClip**, a system designed to bridge this gap. Given a lecture recording, LectureClip transcribes and chunks the lecture into timestamped segments, embeds both the transcript text and sampled visual frames into a shared vector space, indexes these embeddings for fast similarity search, and finally at query time, retrieves and temporally aggregates the most relevant segments into a coherent sub-lecture clip. The system is implemented as a serverless pipeline on AWS, using Amazon Transcribe [3] for speech recognition, Amazon Bedrock Titan [2] for embeddings, and **Amazon Aurora PostgreSQL with pgvector** [1] for vector retrieval.

Our contributions are as follows:

1. We design and implement an end-to-end, cloud-native pipeline for semantic lecture video retrieval that operates at the segment level rather than returning isolated timestamps.
2. We construct a manually annotated evaluation dataset over real university lectures, with fixed queries and ground truth relevant time spans.
3. We evaluate retrieval quality using Recall@k and ablate over two embedding models and three modality configurations, demonstrating that multimodal retrieval outperforms single-modality approaches on visually grounded queries.
4. We discuss practical challenges in chunking strategy, multimodal alignment, and clip assembly that arise specifically in the lecture domain.

2 Related Work

2.1 Commercial Lecture and Video Platforms

Several commercial products address lecture navigation. **Panopto** [17] is widely deployed in universities and provides "Smart Search," which indexes automatic speech recognition (ASR) transcripts and OCR-extracted on-screen text, returning timestamped keyword matches. While effective for exact-term lookup, it does not support semantic queries and requires students to manually inspect each match for relevance. **Echo360** [6] offers similar transcript-based search within its lecture capture ecosystem. **NoteGPT** [15] takes a different approach, generating AI-powered summaries, chapter labels, and notes from uploaded videos. However, it operates outside institutional LMSs and does not support retrieval of specific video segments in response to a user query.

Beyond the education domain, **Twelve Labs** [21] provides a multimodal video search API that accepts natural-language queries and returns relevant moments by jointly reasoning over visual, audio, and textual signals. However, it does not assemble retrieved moments into a coherent instructional sequence, nor does it account for the pedagogical structure (definitions, examples, proofs) characteristic of lectures. **Google NotebookLM** [10] allows users to upload documents and videos and ask questions grounded in the uploaded material, but it returns text-based answers rather than video clips and does not support fine-grained temporal retrieval.

2.2 Video Understanding and Temporal Retrieval

The task of locating a specific moment in a video given a natural-language query is studied under several names in the literature, including *natural language video grounding*, *moment retrieval*, and *video corpus moment retrieval*. **Moment-DETR** [11], developed alongside the QVHighlights benchmark, uses a transformer encoder–decoder architecture to predict moment boundaries and saliency scores jointly. While effective on short, action-oriented clips, it does not address the challenges of hour-long, predominantly static lecture videos.

ColPali [8] introduced a vision-language retrieval model that treats document pages as images, bypassing OCR entirely and matching queries directly against visual page embeddings. This is particularly relevant for slide-heavy lectures where mathematical notation, diagrams, and code listings are poorly handled by text extraction. The **M3AV** dataset [5] provides academic lecture videos with aligned speech transcripts and OCR annotations, including complex formulae, offering a benchmark for evaluating multimodal lecture understanding systems. Similarly, **MaViLS** [4] supplies lectures with explicit slide–video–transcript alignment signals, supporting research on visual segmentation and boundary detection in educational video.

At a higher level, **Video-RAG** [20] proposes a retrieval-augmented generation framework for long-form video question answering. It addresses the problem of chunking videos exceeding one hour without losing cross-segment context, a challenge directly relevant to our project. Their methodology informs our approach to segment-level indexing and retrieval, though we adapt it to the lecture domain where visual change is sparse and content is organized around conceptual topics rather than narrative events.

2.3 Retrieval-Augmented Generation and Embedding Infrastructure

Our system builds on the RAG paradigm, in which a retrieval step supplies relevant context to a language model at generation time [12]. For text embeddings, recent encoders such as E5 [22] and the Amazon Titan Embeddings family [2] have demonstrated strong semantic similarity performance. For visual embeddings, multimodal encoders including CLIP [18] and its successors enable joint text–image retrieval. We use Amazon Bedrock Titan Embeddings for text and explore Bedrock-hosted multimodal models for frame-level visual embeddings.

On the infrastructure side, **LlamaIndex** [13] provides a modular framework for building multimodal RAG pipelines with support for chunking, embedding, and retrieval. **OpenSearch** [16] and **Elasticsearch** [7] both support approximate k-nearest-neighbor search over dense vectors for low-latency retrieval at scale. However, we adopt **Amazon Aurora PostgreSQL with pgvector** [1] as our vector store for its seamless AWS integration and efficient vector similarity search within a relational database. For transcription, **Whisper** [19], OpenAI's open-source ASR model, offers high accuracy across languages and accents; we considered it during design but chose Amazon Transcribe for its managed infrastructure and AWS pipeline integration, as discussed in Section 3.

This landscape reveals a gap at the intersection of lecture-specific content understanding and end-to-end retrieval systems. LectureClip occupies this intersection by combining semantic multimodal retrieval with lecture-aware segmentation and clip assembly within a deployable cloud-native architecture.

3 Technical Approach, Solution and Design

3.1 Requirements

LectureClip transforms long, unstructured lecture recordings into a semantically searchable system that returns temporally grounded video segments. Unlike traditional information retrieval, which returns documents or passages, LectureClip retrieves precise time intervals within a video corresponding to relevant explanations, shifting the retrieval unit from documents to timestamped segments and introducing constraints around temporal precision and coherence.

The system must support video ingestion, automatic transcription, segmentation into meaningful units, and natural-language querying over those units, assembling retrieved segments into a coherent "sub-lecture" suitable for learning. Non-functionally, low query latency is critical for interactivity, while ingestion must scale to long-form content where a single lecture may produce hundreds of segments. Faithfulness is equally important: unlike generative systems, LectureClip must ensure outputs are directly traceable to source content, avoiding hallucination. Finally, the system must remain extensible to accommodate future multimodal models or learned ranking components.

3.2 System Overview

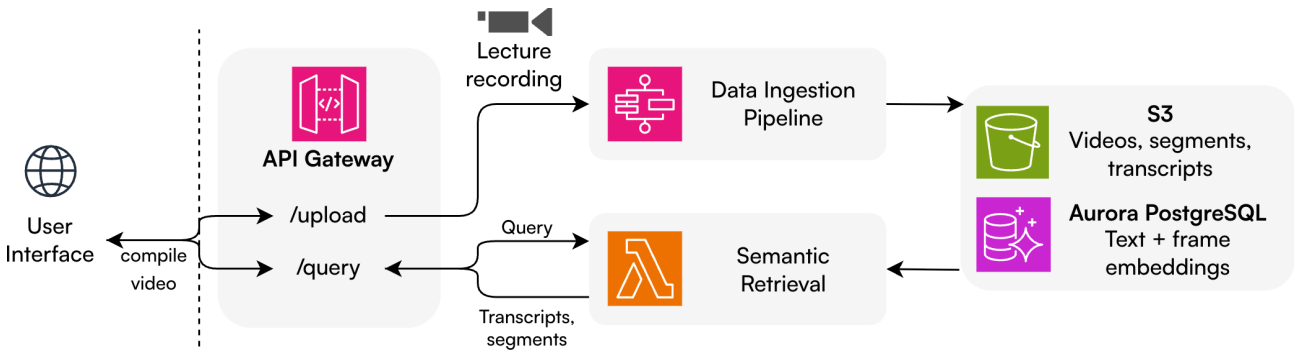

Figure 1: LectureClip's System Architecture

Our system consists of a video ingestion pipeline, a query functionality to retrieve video segments, and a RAG chat. This separation is a central architectural decision that addresses the trade-off between

computational cost and latency requirements. Operations such as transcription and embedding generation are computationally expensive and scale with input size, so performing them at query time would make the system impractical.

This design mirrors classical search systems, where indexing is performed offline and queries are served against precomputed structures. In the context of LectureClip, this separation is particularly important because embedding generation is both expensive and latency-sensitive when performed repeatedly. By precomputing embeddings once during ingestion, our system amortizes this cost across all future queries.

3.3 Ingestion Pipeline

The ingestion pipeline transforms raw video into structured representations suitable for semantic retrieval.

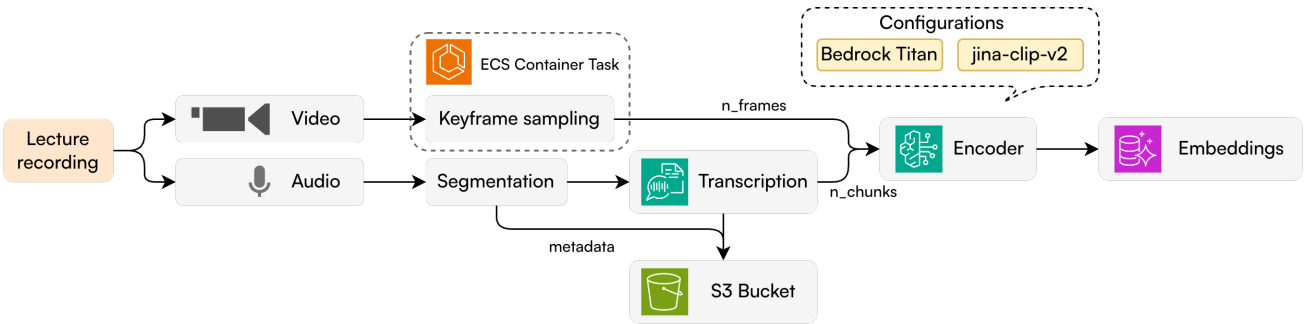

Figure 2: LectureClip's Ingestion Pipeline

The pipeline is defined using AWS Step Functions and is triggered upon a user video upload to Amazon S3. The first stage is transcription, where audio is converted into timestamp-aligned text using an automatic speech recognition (ASR) system. We considered both cloud-based services Amazon Transcribe, a cloud-based service, and Whisper [19], an open-source model by OpenAI. We ultimately chose AWS Transcribe due to its scalability, managed infrastructure, and ease of integration with the broader system. While open-source models may offer greater control and potentially lower long-term cost, they introduce operational complexity and require GPU provisioning, which conflicts with our goal of rapid deployment and maintainability.

Following transcription, the system segments the transcript into retrieval chunks. A key design decision is the segmentation strategy. We adopt a fixed-length approach of every 1000 characters, which provides simplicity and predictable segment sizes, though it may occasionally break semantic continuity by splitting explanations mid-speech or across concept boundaries. In practice, this approach still groups semantically related content within short time windows while preventing segments from becoming too large. The tradeoff is that segment boundaries are heuristic rather than exact, which can introduce minor inaccuracies. However, this is acceptable as retrieval quality depends more on semantic coherence than precise boundaries.

Our system leverages infrastructure supporting two pretrained multimodal models. First, we use **Amazon Titan Multimodal Embeddings G1**, hosted on AWS with on-demand inference pricing. For embeddings, Amazon Titan models are priced at approximately \$0.02 per 1M input tokens (with separate rates for output and batch processing). In addition, we deploy the **jina-clip-v2** model externally on the Modal service [14], configured with a T4 GPU (\$0.59/hour), along with dedicated CPU and memory resources (CPU: \$0.0473/core/hour, Memory: \$0.0080/GiB/hour). The Modal deployment uses a persistent model cache volume, a minimum of one warm container, and a scaledown window of 100 seconds to balance latency and cost.

For video processing, we segment the input and sample a single representative frame within each segment's start and end time. This design choice is motivated by the relatively short segment duration, where additional frame sampling would not yield improved representation. Both segmentation and frame extraction are executed within an ECS task container for scalability and isolation.

Once processing is complete, a final AWS Lambda function, `process-results`, is invoked as part of the Step Functions pipeline. This function generates both frame-level and text-segment embeddings using the configured models and stores them in a unified vector database, enabling efficient multimodal retrieval.

3.4 Query Pipeline

The query pipeline is designed to execute semantic retrieval under strict latency constraints. This is implemented withing a single lambda - `query-segments`

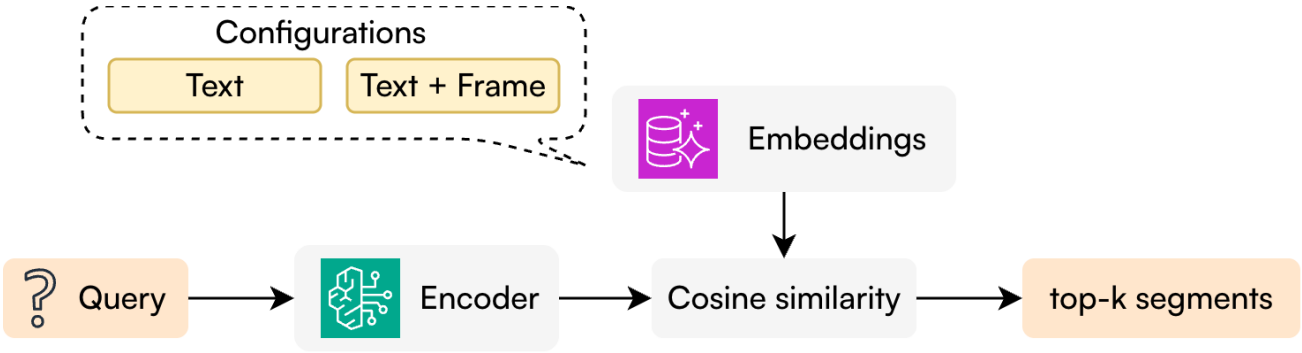

Figure 3: LectureClip's Query Pipeline

At query time, the user's input is first embedded into the same vector space as the segments using the multimodal model from the previous section. This alignment is critical, as it ensures that similarity comparisons are meaningful. We then perform approximate nearest neighbor (ANN) search over the embedding index to retrieve the top-k most similar segments.

We considered both exact and approximate search methods. Exact search guarantees optimal results but scales poorly with the number of segments, making it unsuitable for interactive systems. ANN methods, such as HNSW-based indexing used for our project, provide near-optimal results with significantly lower latency, making them a better fit for our requirements. This reflects a common tradeoff in retrieval systems between optimality and performance, where slight reductions in accuracy are acceptable in exchange for substantial gains in speed.

Another important design decision is to scope retrieval to a single lecture. This significantly reduces the search space and improves latency, while also simplifying ranking. However, this comes at the cost of limiting cross-lecture reasoning, which may be desirable in future iterations. This tradeoff reflects a prioritization of performance and simplicity over global retrieval capabilities.

Currently, segments are ranked using cosine similarity. While this is effective as a baseline, it does not capture more complex relationships between query and segment representations, particularly in a multimodal setting. To address this, we propose extending the system with a learned ranking model that jointly considers text and visual embeddings. Such a model could learn to weight different modalities and contextual signals, improving retrieval quality beyond what is possible with simple distance metrics.

After retrieval, the backend returns an ordered list of timestamped segments rather than constructing a new video artifact. These segments are sorted temporally and passed to the frontend, which is responsible for orchestrating playback as a contiguous experience. This design choice shifts aggregation from the backend to the client, allowing the frontend to provide richer user controls such as replaying individual segments, skipping ahead, or overriding the current playback position.

This approach improves maintainability and reduces backend complexity, as no additional video processing or re-encoding is required. It also enhances user experience by preserving interactivity and flexibility in navigation. However, it introduces a tradeoff: because segments are not physically merged, coherence depends on the quality of segmentation and ordering, and users may still encounter minor discontinuities or irrelevant transitions if segment boundaries are imprecise.

3.5 RAG Chat

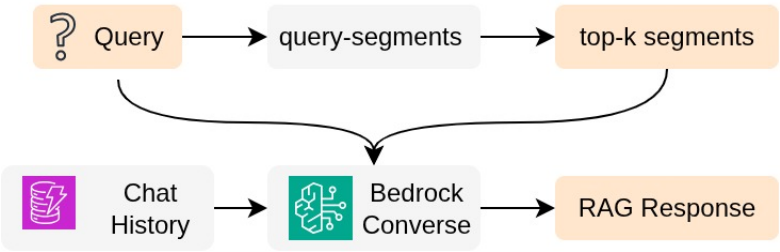

Figure 4: RAG architecture

We implement a retrieval-augmented generation (RAG) chat system that enables users to interact with lecture content through an LLM grounded in precomputed embeddings. The system is deployed as an AWS Lambda function exposed via an API endpoint. When a user submits a query, it is first converted into a dense vector embedding using one of two multimodal models: **Amazon Titan Multimodal Embeddings G1** (via AWS Bedrock) or the externally hosted **jina-clip-v2** model on Modal.

The resulting query embedding is used to perform a similarity search over stored embeddings in an **Amazon Aurora PostgreSQL** database with **pgvector** support. This retrieves the most relevant transcript segments and associated metadata (e.g., timestamps and text), based on cosine similarity.

The retrieved segments are then incorporated into the prompt as contextual grounding for the language model. To support multi-turn interactions, prior conversation history is stored and retrieved from **Amazon DynamoDB**, allowing the system to maintain context across user queries. The prompt is structured to encourage grounded responses with explicit references to retrieved segments.

Finally, the augmented query is passed to a large language model hosted on **AWS Bedrock**, (**Claude (Haiku 4.5)** is used for this), to generate a response. The system returns both the generated answer and relevant segment metadata to the user, while persisting the updated conversation state in DynamoDB for subsequent interactions.

3.6 Data Model

The data model is centered around representing lectures as collections of timestamped segments, with supporting tables for evaluation, user management, and vector search. The schema is defined in an idempotent migration Lambda (`db-migrate`) that executes DDL statements on each deployment, ensuring consistency across environments. Listing 1 shows the three core tables.

The `lectures` table stores per-recording metadata and an S3 video URI, while `segments` decomposes each lecture into indexed, non-overlapping time intervals with associated transcript text. The composite unique constraint on `(lecture_id, idx)` ensures segment ordering is unambiguous, which the frontend relies on when mapping retrieved indices back to temporal boundaries for playback.

The key design decision in the embedding layer is to decouple embeddings from segments via a separate `segment_embeddings` table rather than adding vector columns directly to `segments`. As shown in Listing 1, each embedding row carries a `model_id` and an `is_frame_embedding` flag, allowing multiple embedding variants per segment to coexist. This is essential for our ablation experiments, where the same segment may have embeddings from both **Amazon Titan** and **jina-clip-v2**, in both text and frame modalities. An HNSW index with `vector_cosine_ops` enables approximate nearest-neighbor search at query time over the 1024-dimensional vectors.

The schema also includes tables supporting evaluation and user management, omitted from Listing 1 for brevity. The `eval_queries` and `truth_spans` tables store fixed natural-language queries and their ground-truth relevant time intervals with ordinal relevance labels, enabling reproducible computation of Recall@k without relying on external annotation files. The `eval_task_runs` and `sublecture_ratings` tables capture user study outcomes, including task completion times and Likert-scale judgments on relevance, coverage, coherence, and redundancy. A `users` table tracks registered Cognito accounts; a

Listing 1: Core relational schema (simplified).

```
CREATE TABLE lectures (
    lecture_id  UUID PRIMARY KEY,
    course_id   TEXT,
    title       TEXT,
    duration_s  REAL,
    video_uri   TEXT        NOT NULL,
    user_id     UUID        REFERENCES users(user_id),
    ingested_ts TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE segments (
    segment_id UUID PRIMARY KEY,
    lecture_id UUID NOT NULL REFERENCES lectures(lecture_id),
    idx        INT  NOT NULL,
    start_s    REAL NOT NULL,
    end_s      REAL NOT NULL,
    text       TEXT NOT NULL,
    UNIQUE (lecture_id, idx)
);

CREATE TABLE segment_embeddings (
    embedding_id       UUID         PRIMARY KEY,
    segment_id         UUID         NOT NULL
                       REFERENCES segments(segment_id),
    embedding          vector(1024) NOT NULL,
    model_id           TEXT         NOT NULL,
    is_frame_embedding BOOLEAN      NOT NULL DEFAULT false,
    created_ts         TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE INDEX segment_embeddings_hnsw_idx
    ON segment_embeddings
    USING hnsw (embedding vector_cosine_ops);
```

backfill migration retroactively associates existing lectures with users by extracting email addresses from S3 URI paths.

We use **Amazon Aurora PostgreSQL** with pgvector as a unified store for both relational metadata and vector embeddings. While dedicated vector databases such as OpenSearch may offer better scalability at very large scale, co-locating embeddings with their associated segment metadata in a single relational system simplifies joins, reduces operational overhead, and allows transactional consistency across ingestion steps. This tradeoff is appropriate for our expected workload of tens of lectures per course, where index sizes remain well within Aurora's capacity.

3.7 Scalability and Performance

The system achieves scalability through a combination of event-driven orchestration and bounded query-time computation. The entire backend is implemented in a **serverless architecture using AWS Lambda**, enabling automatic scaling and eliminating the need for persistent infrastructure management. The ingestion pipeline is defined using **AWS Step Functions** and is triggered asynchronously upon video upload to **Amazon S3**. This design decouples expensive operations such as transcription (via **Amazon Transcribe**), segmentation, frame extraction (via **ECS**), and embedding generation from query execution. These steps can be parallelized across segments, allowing ingestion throughput to scale with available compute resources.

At query time, computation is intentionally lightweight, consisting primarily of a single embedding generation using either **Amazon Titan Multimodal Embeddings G1** or the externally hosted **jina-clip-v2** model on Modal, followed by a similarity search over **pgvector** in **Amazon Aurora PostgreSQL**. Response generation is handled by **Claude (Haiku 4.5)** via **AWS Bedrock**. This design ensures that query latency remains stable regardless of lecture length, as retrieval operates over an indexed vector space rather than raw data size. Additionally, restricting retrieval to segments associated with a single lecture further reduces the effective search space and improves performance.

From a scalability perspective, the self-hosted **jina-clip-v2** model on Modal is better suited for high-throughput workloads, as it can dynamically scale the number of containers used for inference based on demand. In contrast, the on-demand **Amazon Titan** embedding model is subject to throughput limits (TPS), which can constrain performance under heavy query loads.

However, several bottlenecks remain. Embedding generation cost scales linearly with the number of segments, making ingestion expensive for long videos, particularly when generating both text and frame embeddings. The size of the vector index in Aurora also grows with the dataset, which may impact storage, memory usage, and retrieval latency over time. The use of multimodal embeddings further increases both computational and storage overhead. These limitations suggest opportunities for future optimization, such as hierarchical indexing, more efficient vector compression, or selective embedding strategies that reduce redundancy while preserving retrieval quality.

3.8 Design Tradeoffs and Limitations

The system reflects several key design tradeoffs. Our use of a retrieval-augmented generation (RAG) approach prioritizes **faithfulness and interpretability**, ensuring that responses from **Claude (Haiku 4.5)** are grounded explicitly in retrieved transcript segments. This improves reliability compared to purely generative approaches, but introduces some overhead in prompt construction and dependency on retrieval quality.

The decision to use **segment-level indexing** with both text and frame embeddings improves retrieval precision and enables multimodal search, particularly for visually grounded queries. However, this increases preprocessing cost, as embeddings must be generated using models such as **Amazon Titan Multimodal Embeddings G1** and the open-source **jina-clip-v2**, and also leads to higher storage requirements. Notably, the self-hosted jina model is a relatively small ($\sim$ 0.9B parameter) model, which may not perform as well as larger proprietary models like Amazon Titan in terms of embedding quality and retrieval accuracy. Similarly, restricting retrieval to a single lecture reduces latency and simplifies ranking, but limits the system's ability to perform cross-lecture knowledge aggregation.

From an infrastructure perspective, adopting a fully **serverless architecture with AWS Lambda** and orchestrating ingestion via **Step Functions** improves scalability and maintainability, but introduces tradeoffs in terms of cold start latency and reduced control over long-running tasks.

Several limitations remain. Segment boundaries are determined heuristically and may not perfectly align with semantic or conceptual units. The system does not currently support cross-lecture retrieval or global indexing across multiple videos. While multimodal embeddings improve recall, they also introduce additional computational and storage overhead.

Future work will focus on addressing these limitations through hierarchical or adaptive segmentation, hybrid retrieval methods that combine keyword-based and semantic search, cross-lecture indexing, and more efficient embedding strategies. Further improvements to the RAG pipeline, including better prompt optimization and response synthesis, could enhance the balance between retrieval accuracy, system efficiency, and overall user experience while preserving strong grounding in source material.

4 Experiments and Results

4.1 Evaluation Setup

Task formulation. We evaluate the retrieval quality of LectureClip with a segment-level ranking task. Given a natural language query, the system is evaluated on its ability to return the $k = 3$ segments most relevant to that query. This effectively serves as a holistic integration test of the ingestion and query pipelines.

Dataset. We construct an evaluation set of 8 (query, lecture) pairs drawn from two recorded Computer Science lectures: a CSC420 lecture on radiometry and photometric cues, and a CSC111 embedded ethics lecture on privacy and contact tracing. Table 1 details the queries used. Each query is designed to reflect realistic student search behaviour, and they are further stratified by type: text-based queries

whose ground truth segments are recoverable what's spoken, and visual-based queries whose ground truth segments depend on visual content on the slides, such as diagrams and illustrations. For each query we manually annotate a ground truth set $\mathcal{G}$ of three relevant segments (i.e. that collectively provide sufficient information to answer the query), ordered by decreasing relevance.

ID	Query	Lecture	Type	Ground Truth
Q1	What is the unit of radiance, and how is it different from irradiance?	CSC420	Text	44, 39, 41
Q2	How do diffuse and specular surfaces appear differently?	CSC420	Visual	59, 60, 61
Q3	What does it mean for a surface to be Lambertian?	CSC420	Text	45, 58, 83
Q4	What is the BRDF?	CSC420	Text	47, 58, 65
Q5	Is it fair to decide which contact tracing app to use by vote?	CSC111	Text	20, 13, 22
Q6	How do the two contact tracing designs differ in what they reveal?	CSC111	Visual	12, 1, 8
Q7	Why doesn't anonymization always protect privacy?	CSC111	Text	2, 3, 12
Q8	How does the Bluetooth exposure notification system work?	CSC111	Visual	7, 8, 9

Table 1: Evaluation dataset. Ground truth segment indices are listed in order of decreasing relevance.

Metric. We adopt Recall@k as our evaluation metric, defined as

$$\text{Recall@}k \;=\; \frac{|\,\text{Ret}_k \cap \mathcal{G}\,|}{|\mathcal{G}|}$$

where Ret_k is the set of top-k retrieved segment indices. Because LectureClip reorders retrieved clips chronologically before playback regardless of retrieval score Recall@3 is the most relevant metric as it measures solely coverage of the ground truth segments without penalizing the ordering at which they are recovered.

Ablation. We ablate over two variables: embedding model (Jina-clip-v2 vs. Amazon Bedrock Titan Embeddings) and "retrieval mode". For each segment we precompute a text embedding θ_{text} and a frame embedding θ_{frame} during ingestion; the retrieval score against a query q is thus

$$s(q,c) \;=\; \alpha\,\theta_{\text{text}}(q,c) \;+\; \beta\,\theta_{\text{frame}}(q,c), \tag{1}$$

with $(\alpha,\beta) \in \{(1,0),(0.5,0.5),(0,1)\}$ corresponding to text-only, text+frame, and frame-only "retrieval modes". This gives six configurations in total.

4.2 Results

Table 2 reports Recall@3 for all six configurations.

First, we see that text-only retrieval performs best on queries whose answers are fully expressed in speech. Jina text-only and Titan text+frame both achieve the highest text-query average (0.47), while frame-only retrieval scores near zero for both models on these queries. This suggests that slide embeddings carry little information about the verbal content of surrounding segments, as adding frame embeddings alongside text provides no benefit for Jina on text queries, and only marginal benefit for Titan (0.47 vs. 0.27). Conversely, for visual-content queries, combining text and frame embeddings outperforms either modality alone: Jina text+frame achieves the highest visual-query average (0.78) versus frame-only (0.44) and text-only (0.67), as frame embeddings capture content absent from transcripts while text embeddings link queries to the speech. This benefit is less pronounced for Titan (text+frame: 0.56, frame: 0.56, text: 0.33), consistent with its weaker cross-modal performance relative to Jina-clip-v2 (Jina: 0.54 vs. Titan: 0.50 overall).

The visual category is not uniformly frame-dependent, however. Q8 (Bluetooth exposure notification) achieves perfect text-only retrieval for both models because the transcript fully narrates the illustrated mechanism. Q2 (diffuse vs. specular surface appearances), by contrast, relies on photographs never

Model	Modality	Text R@3	Visual R@3	Overall R@3
Jina-clip-v2	Frame	0.20	0.44	0.29
Jina-clip-v2	Text+Frame	0.40	**0.78**	**0.54**
Jina-clip-v2	Text	**0.47**	0.67	**0.54**
Titan	Frame	0.27	0.56	0.38
Titan	Text+Frame	**0.47**	0.56	0.50
Titan	Text	0.27	0.33	0.29

Table 2: Recall@3 averaged over text-based ($n = 5$) and visual-based ($n = 3$) evaluation queries. Bold indicates the best result per column.

described verbally, so Titan text scores 0 while Jina text+frame reaches 0.67. This distinction matters practically: the value of frame embeddings scales with how much relevant content is expressed visually rather than verbally.

Independent of modality and model, queries whose relevant segments are spread far apart perform consistently poorly. No configuration exceeds Recall@3= 0.33 on Q1 or Q5, both of which have ground-truth segments separated by large gaps. A single embedding matched against individual segments tends to retrieve the closest local match rather than the combination of distant segments that together answer the query. This limitation is not addressable by changing model or modality alone, and points toward hierarchical or multi-step retrieval as future work.

Taken together, these results suggest using Jina-clip-v2 with text+frame embeddings as the default configuration: it matches text-only performance on speech-recoverable queries while providing a substantial advantage on visually dependent ones.

4.3 User Study

To complement the offline retrieval evaluation, we conducted a qualitative user study.

Participants and procedure. We recruited six Computer Science undergraduates with prior experience studying from recorded lectures. Each participant completed two study tasks: one with LectureClip and one with a baseline condition using manual video scrubbing and transcript keyword search in the browser. In each task, participants had to find and review material relevant to a given study question within a 60-minute lecture, with a 10-minute time limit. Afterward, they completed a post-task questionnaire on 5-point Likert scales and a brief semi-structured interview.

Results. Table 3 summarises the Likert-scale responses across four dimensions. Participants rated LectureClip favourably on all four measures, with the strongest agreement on time efficiency (mean 4.3) and ease of locating relevant content (mean 4.2). Clip coherence received the lowest average rating (mean 3.7), consistent with the segmentation limitations discussed in Section 3.

Statement	Mean	SD
The system helped me find relevant content faster than manual search.	4.3	0.65
The retrieved clips were relevant to my query.	4.1	0.79
The assembled clips formed a coherent explanation.	3.7	0.89
I would use this tool regularly for exam review.	4.2	0.72

Table 3: User study Likert-scale results ($n = 12$; 1 = Strongly Disagree, 5 = Strongly Agree).

On average, participants located sufficient material to answer their study question in 4.2 minutes using LectureClip, compared to 7.8 minutes under the baseline condition, a 46% reduction in search time. All 6 participants successfully identified at least one relevant segment using LectureClip within the time limit, compared to 4 out of 6 under the baseline.

Qualitative feedback. Participants mainly valued that LectureClip let them ask conceptual questions without needing to guess exact keywords. They found this useful because keyword search often failed when instructors paraphrased or used informal wording, while LectureClip's semantic retrieval could still find relevant lecture segments. The main weakness participants mentioned was that some assembled clips felt disjointed when segment boundaries split a single explanation. A few also noted that retrieval was sometimes only loosely related to the query, pointing to room for improvement in ranking precision. Overall, the study suggests LectureClip is more effective than manual navigation and keyword search for realistic study tasks, while highlighting segmentation and ranking as the main areas for improvement.

5 Future Work and Next Steps

Our evaluation reveals several concrete limitations of the current system, each of which suggests a specific line of future development.

Adaptive segmentation. The fixed-length chunking strategy (1000 characters) does not respect semantic boundaries, occasionally splitting explanations across chunks or merging unrelated topics. Our user study reinforces this, as clip coherence received the lowest satisfaction rating and participants noted discontinuities at segment boundaries. A hybrid strategy that uses slide transitions as primary boundaries and falls back to text-based splitting within long single-slide segments would better align retrieval units with lecture structure. Additional signals such as ASR pause duration and embedding drift across consecutive windows could further improve boundary detection.

Hierarchical and multi-step retrieval. Queries whose ground truth segments are spread across distant portions of a lecture (e.g., Q1 and Q5) perform consistently poorly under single-pass retrieval, regardless of model or modality. A hierarchical approach could first identify relevant topics at a coarse level, then retrieve segments within those regions. Alternatively, iterative query refinement based on initially retrieved segments could help recover complementary content that collectively answers a question even when no single segment is sufficient.

Learned ranking and modality fusion. The current fixed linear combination of text and frame similarities does not adapt to query characteristics. A learned ranking model could dynamically weight modalities based on query type, for instance upweighting frame similarity for queries referencing diagrams. Cross-encoder reranking over the top-k candidates offers another path to improved precision at modest computational cost.

Cross-lecture retrieval and hybrid search. The system currently restricts retrieval to a single lecture, but students often need to locate content across an entire course. Extending to a global index introduces challenges around search space, latency, and result disambiguation. Additionally, combining dense vector search with sparse keyword matching (e.g., BM25) via reciprocal rank fusion could better handle exact technical terminology common in STEM lectures.

Expanded evaluation and embedding efficiency. Our offline evaluation covers only eight queries over two lectures. A broader dataset spanning more disciplines and lecture styles would provide more robust quality estimates, and a longitudinal study tracking usage patterns over a full term would yield stronger evidence of impact on learning outcomes. On the infrastructure side, selective embedding strategies that skip visually redundant frames, quantized embeddings, and model distillation could reduce ingestion cost and improve viability for institution-wide deployment.

6 Team Reflection

LectureClip began as a broad idea of "make lecture videos searchable" and our first major challenge was scoping the required tools for this project. The landscape analysis forced us to find where existing tools fall short, such as keyword-only retrieval or no clip assembly, and what a meaningful improvement would look like. By analyzing competitors, research papers, existing tools and infrastructure components side by side, we avoided building features that already exist and focused on the semantic retrieval gap.

The most significant technical lesson was the importance of data pipeline reliability over model sophistication. Early in the project, we invested substantial time exploring advanced VLMs and multimodal embeddings, only to discover that our primary blocker was the inefficiency of the underlying data pipeline. One early design decision was to use fixed-length segmentation for simplicity, which provided predictable chunk sizes but occasionally fails to preserve semantic structure. In practice, this can lead to explanations being split across segments or unrelated content being grouped together, which can reduce both retrieval precision and user comprehension. While we retained this approach in our final system for its simplicity and predictability, the experience highlighted the importance of adaptive segmentation as a key direction for future improvement.

A second issue emerged in the ingestion pipeline, where each upload triggered a full transcription and embedding workflow regardless of whether the same lecture had already been processed. This resulted in redundant computation and high latency, particularly for large lecture videos. Our initial solution used content-based hashing to detect duplicate uploads, but computing hashes over large files introduced significant overhead and became a bottleneck itself. We ultimately replaced this with a file name-based encoded identifier, which, while less robust, dramatically reduced processing time and covered the majority of real-world duplicate-upload scenarios. This change reflected a practical tradeoff between theoretical correctness and system performance.

Finally, we encountered a critical encoding issue in the retrieval pipeline, where inconsistencies between URL-encoded and raw identifiers caused failures in segment lookup despite the data existing in storage. This bug was difficult to diagnose because individual components behaved correctly in isolation, and failures only appeared in end-to-end execution. We resolved this by instrumenting identifier flow across the system, introducing controlled test cases, and standardizing encoding at all system boundaries. This fix restored retrieval consistency and reinforced the importance of strict data contracts and normalization in distributed systems.

Connection to course material. The course content informed most of our design decisions.

The modules on **Docker, Kubernetes, cloud infrastructure, and Terraform** influenced our choice of a serverless AWS architecture. We used Lambda, API Gateway, S3, and Aurora RDS as managed services to avoid operational overhead, following the infrastructure-as-code philosophy discussed in class.

The **model serving** lectures helped us weigh the trade-offs between self-hosting Whisper with GPU costs and quantization concerns versus using Amazon Transcribe as a managed service. We chose the latter for reliability and cost predictability at our scale.

The **evaluation and metrics** lecture guided our benchmark design. We adopted Recall@k because LectureClip reorders clips chronologically before playback, making rank-sensitive metrics inappropriate. The course's emphasis on end-to-end evaluation also motivated our user study and the identification of expanded evaluation as a future direction.

The **feature stores** lecture shaped our data schemas. Our PostgreSQL tables for segments, embeddings, and evaluation labels function analogously to a feature store, with representations written once at ingestion and read many times at query time. Our separation of storage backends—S3 for raw video, RDS for metadata, and RDS with pgvector for embeddings—follows the principle of choosing the right store for each data modality.

The **scalability** lecture helped us reason about pipeline cost at scale. It also introduced us to RAG and confirmed that precomputing and caching embeddings at ingestion time, rather than computing them at query time, is essential for a responsive and cost-effective deployment.

References

[1] Amazon Web Services. Amazon aurora postgresql-compatible edition with pgvector. https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.VectorDB.html, 2024. Accessed: 2026.

[2] Amazon Web Services. Amazon Titan embeddings models. https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html, 2024. Accessed: 2025.

[3] Amazon Web Services. Amazon Transcribe. https://aws.amazon.com/transcribe/, 2024. Accessed: 2025.

[4] Katharina Anderer, Andreas Reich, and Matthias Wölfel. MaViLS, a benchmark dataset for video-to-slide alignment, assessing baseline accuracy with a multimodal alignment algorithm leveraging speech, OCR, and visual features. In *Interspeech 2024*, pages 1375–1379, 2024.

[5] Zhe Chen, Heyang Liu, Wenyi Yu, Guangzhi Sun, Hongcheng Liu, Ji Wu, Chao Zhang, Yu Wang, and Yanfeng Wang. M^3AV: A multimodal, multigenre, and multipurpose audio-visual academic lecture dataset. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 9041–9060, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[6] Echo360. Echo360 active learning platform. https://echo360.com, 2024. Accessed: 2025.

[7] Elastic. Elasticsearch: The official distributed search and analytics engine. https://www.elastic.co/elasticsearch, 2024. Accessed: 2025.

[8] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models. *arXiv preprint arXiv:2407.01449*, 2024.

[9] Michail N. Giannakos, Letizia Jaccheri, and John Krogstie. Exploring the relationship between video lecture usage patterns and students' attitudes. *British Journal of Educational Technology*, 47(6):1259–1275, 2016.

[10] Google. NotebookLM. https://notebooklm.google.com, 2024. Accessed: 2025.

[11] Jie Lei, Tamara L. Berg, and Mohit Bansal. QVHighlights: Detecting moments and highlights in videos via natural language queries. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[13] LlamaIndex. LlamaIndex: Data framework for LLM applications. https://github.com/run-llama/llama_index, 2024. Accessed: 2025.

[14] Modal Labs. Modal: High-performance cloud for AI and ML. https://modal.com/, 2024. Accessed: 2025.

[15] NoteGPT. NoteGPT: AI-powered video summaries and notes. https://notegpt.io, 2024. Accessed: 2025.

[16] OpenSearch Project. OpenSearch. https://opensearch.org, 2024. Accessed: 2025.

[17] Panopto. Panopto video platform. https://www.panopto.com/features/, 2024. Accessed: 2025.

[18] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR, 2021.

[19] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International*

Conference on Machine Learning (ICML), pages 28492–28518. PMLR, 2023.

[20] Xubin Ren, Lingrui Xu, Long Xia, Shuaiqiang Wang, Dawei Yin, and Chao Huang. VideoRAG: Retrieval-augmented generation with extreme long-context videos. *arXiv preprint arXiv:2502.01549*, 2025.

[21] Twelve Labs. Twelve labs video understanding API. https://docs.twelvelabs.io, 2024. Accessed: 2025.

[22] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.